LLDD FE - Integration Contracts

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน
และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	FE
Estimate	16 ชั่วโมง (ไม่มี unit test แยก — ดูเหตุผลใน NO_UNIT_TEST_DOCS)
Owner	Chidchanok <lin> Saengamnat
Target repository	`SBP/srm-sps-spsap-web-frontend` (sbp-portal · Next.js · `NEXT_PUBLIC_APP_TARGET=sbpm`) — เรียก API ผ่าน `SBP/srm-sps-spsap-sbp-bff` เท่านั้น ห้ามยิง store-backend ตรง
Objective	กำหนดสัญญากลางฝั่ง Frontend สำหรับการ consume API ทุกหน้า: auth/session, error handling, pagination, format, document action และ RBAC/menu gating

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ใช้ endpoint ของ	LLDD-BE-API-Document-Workflow-Actions.pdf	`POST /api/v1/sgi/document/{docNo}/actions`
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Integration-SBP-Platform.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

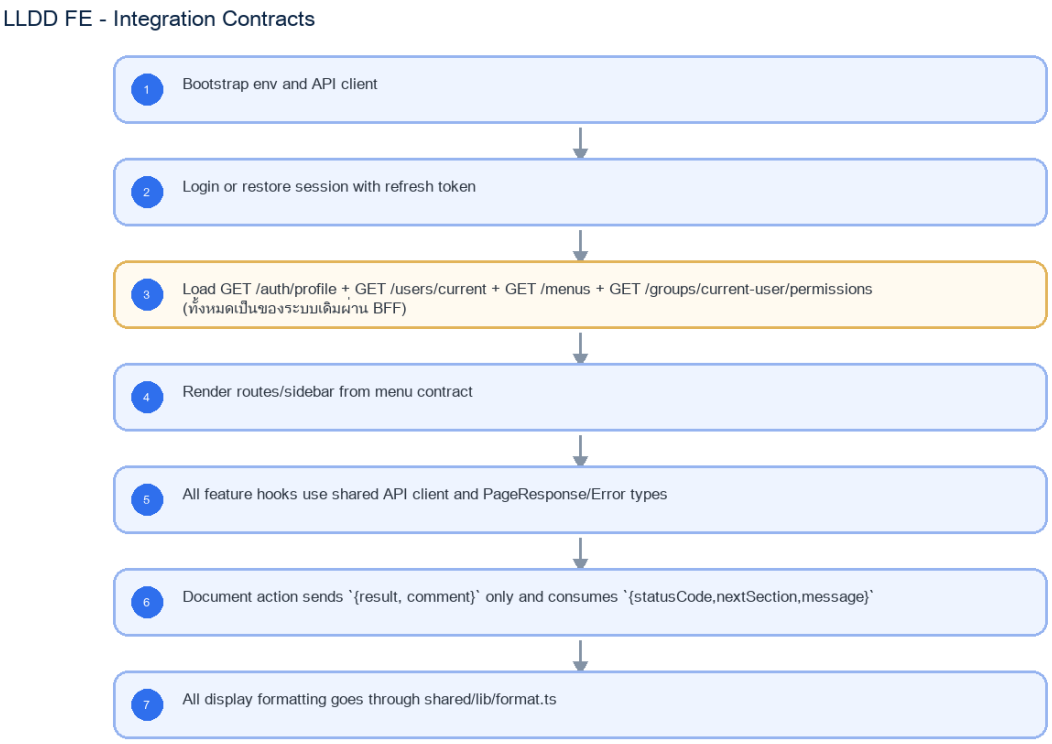
- Shared API client contract
- Auth/JWT consumption from platform reference
- Error display and validation message mapping
- Date/year/money/docNo formatting
- Pagination, list empty/loading/error state
- Document action result enum and response consumption
- RBAC/menu gating and editable section flags

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

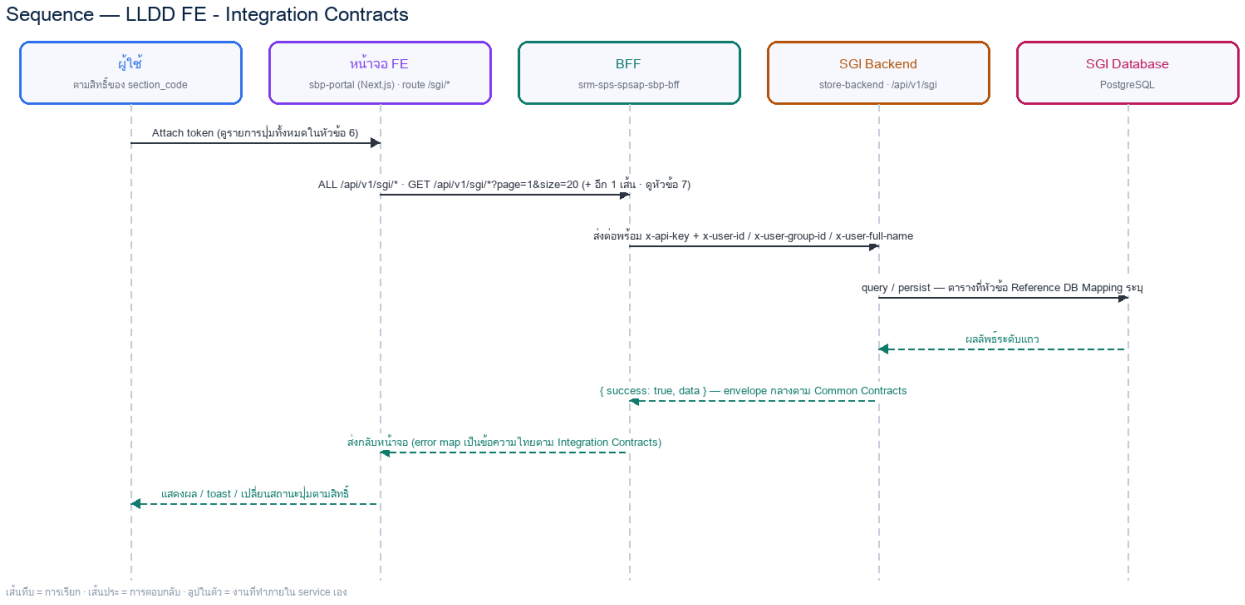
4.1 Implementation Flow (ลำดับขั้นการทำงาน)



รูปที่ 1: Implementation flow reference: LLDD FE - Integration Contracts

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ



รูปที่ 2: Sequence diagram: LLDD FE - Integration Contracts

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
Authorization	Bearer JWT	required except /auth/login and /auth/refresh	แนบโดย axios interceptor เท่านั้น; component ห้าม set header เอง
ApiError	{code,message}	message required	แสดง message จาก BE ตรง ๆ; fallback ใช้เฉพาะ network/no response
PageResponse<T>	{page,size,total,items}	page>=1 size<=100	ใช้กับ DataTable/Pager ทุกหน้า
date/month	ISO ค.ศ. YYYY-MM-DD / YYYY-MM	payload uses CE	แสดงผ่าน formatDateThai/formatMonthThai จุดเดียว — ค่าเริ่มต้นเป็น ค.ศ.
docNo	YYYY/xxxxx ค.ศ.	do not split except route params	route ใช้ /documents/:year/:running แล้วประกอบ docNo
result	actionOptions[].value (ไม่ใช่ label)	required before submit action	ส่งเป็น payload `{result, comment}` เท่านั้น · `result` = `actionOptions[].value` — `label` เป็นข้อความบนปุ่มที่ยาวกว่า (เช่น label ส่งเจ้าหน้าที่ SBP DSA ดำเนินการ → value ส่งเจ้าหน้าที่ SBP DSA) ส่ง label ไปจะไม่ตรง 7 ค่าของ enum
ActionResponse	{statusCode,nextSection,message}	required after action	invalidate detail/timeline/tasks แล้ว resolve label จาก /sgi/lookup/document-statuses
envelope	{success, data, requestId}	BFF unwrap ของ store-backend แล้ว 1 ชั้น	FE อ่าน `data.data` ชั้นเดียว (ยืนยันจากโค้ด BFF จริง 2026-09-04 · รายละเอียดเต็มอยู่ใน LLDD-BE-Integration-SBP-Platform.pdf หัวข้อ 5)
MenuItem	{menuCode,label,route,group}	จาก GET /menus + GET /groups/current-user/permissions ของระบบเดิม (ผ่าน BFF)	sidebar filter ด้วย menuCode จาก API; ไม่ hardcode role
canEditSections	string[]	from document detail	ใช้เปิด/ปิด section editor; FE ไม่คำนวณสิทธิ์เอง

5.80 Namespace + กลุ่ม path ของงานประกันรายได้ (มติ 2026-08-25)

SGI ไม่ได้แยก backend/พอร์ทัลใหม่ (มติ DP-10 ให้อยู่ใน `srm-sps-spsap-store-backend` และโมดูลใน `srm-sps-spsap-web-frontend` เดิม) ทุกอย่างของงานประกันรายได้จึงอยู่ใต้ ชื่อเดียวกันทั้ง 3 ชั้น แล้วแตกเป็น 5 กลุ่มย่อยตามกลุ่มงาน — ตรงกับ 5 กลุ่มใน `API design` แบบ 1:1 (เดิม 6 กลุ่ม · กลุ่ม `/sgi/interface/*` ถูกตัดทิ้งกลุ่ม 2026-09-22)

ชั้น	รูปแบบ	ตัวอย่าง
URL ของ API	/api/v1/sgi/<กลุ่ม>/<resource>	`/api/v1/sgi/document/{docNo}/actions`
route ของหน้าจอ	/sgi/<กลุ่ม>/<หน้า>	`/sgi/documents/waiting` · `/sgi/report/status-summary` — • กลุ่มเอกสารบนหน้าจอเป็นพหูพจน์ `documents` ต่างจาก API ที่เป็นเอกพจน์ `document` (ยืนยันกับทะเบียนเมนูของ ABS แล้ว 2026-09-22 · `hasPermission()` เทียบแบบ exact match สะกดผิดตัวเดียว = AccessDenied ทั้งหน้า)
โฟลเดอร์ไฟล์	`**/sgi/*`	`src/app/(main)/sgi/*` · `src/services/sgi/*` · `src/types/sgi/*`

5 กลุ่มย่อยใต้ `sgi`

กลุ่ม	prefix	เส้น	ครอบคลุมอะไร
งาน & เอกสารประกันรายได้	`/sgi/document/*`	11	`/tasks` (กล่องงาน) · ค้นหา/สร้าง/แก้เอกสาร · `/{docNo}/actions` · `/timeline` · `/attachments` · `/sales`
ข้อมูลอ้างอิง (Lookup)	`/sgi/lookup/*`	2	`/document-statuses` · `/workflow-sections` — อ่านอย่างเดียว ไม่มีหน้าจอดูแล
Master Data	`/sgi/master/*`	9	`/factors` (CRUD 4) · `/competitors` (CRUD 4) — master ที่มีหน้าจอดูแลของตัวเอง · `/user-import` นำเข้าผู้ใช้งานจาก CSV (BFF เรียกเองจากหน้า /uploads)
รายงาน	`/sgi/report/*`	2	`/status-summary` · `/status-summary/export`
Workflow ภายใน	`/sgi/workflow/*`	3	`/instances` · `/instances/{id}` · `/summary`

Batch job ไม่มีกลุ่ม path ของตัวเอง — Jobs 2, 3, 4, 5, 6, 7, 8, 8b, 9, 10, 11, 12 (รวม 12 job) รันด้วย AWS Batch/CLI ไม่ได้เปิด endpoint (กลุ่ม Batch Job Admin 6 เส้นถูกตัดทิ้ง 2026-08-06 · กลุ่ม `/sgi/interface/*` ถูกตัดทิ้ง 2026-09-22) · หน้าต่างที่มองเห็นผลของ job คือ ตาราง `sgi_interface_transactions` โดยตรง (อ่านด้วย SQL) · `integration_log` ของระบบ SBP เดิม · อีเมลเตือนของ Job 10 และ application log

repo ปลายทางของ batch job ทุกตัวคือ SBP/`srm-sps-spsap-sop-sgi-batch` — วิธีลงทะเบียน job, รูปแบบ `INPUT`, publisher ของ RabbitMQ, `S3Service` และ `integration_log` อ่านได้ที่ `srm sps spsap sop sgi batch` · ห้ามออกแบบ batch ของ SGI โดยไม่อ่านไฟล์นั้นก่อน · job ที่ถูก สั่งด้วยไฟล์ที่มาถึง S3 (Job 2 · 3 · 5 · มติ 2026-10-01) ใช้ EventBridge rule → `SubmitJob` ชุดตั้งค่าอยู่ใน `output/aws/sgi-s3-triggers/`

ทำไมต้องมี prefix (ไม่ใช่แค่ความสวยงาม)

ระบบเดิมมีอยู่แล้ว	ของ SGI ถ้าไม่ใส่ prefix	ผล
`/document` · `/statement/...`	`/documents`	ชนเชิงความหมาย อ่าน routing แล้วสับสน
`/report` · `/performance-report` · `/statement/report/ej`	`/reports/status-summary`	ชนเชิงความหมาย
`/interface/sta/upload-cmadd` · `/interface/add`	`~~/interfaces/sta/ack~~`	✓ หมดความเสี่ยงแล้ว — เส้นของเราถูกตัดเมื่อ 2026-09-08
`/common` · `/master` · `/store`	`/factors` `/competitors` `/document-statuses`	ปนกับ master ของโมดูลอื่น

- ฝั่ง NestJS: `SgiModule` เดียว ผูก prefix ที่ระดับโมดูล (`RouterModule.register([{ path: 'sgi', module: SgiModule }])`) แล้วแตกเป็น 6 controller ตามกลุ่ม (`DocumentController` `LookupController` `MasterController` `ReportController` `WorkflowController` `InterfaceController`) — ห้ามเติม `sgi/` ในแต่ละ `@Controller()`
- ในกลุ่ม `document` ต้องประกาศ route คงที่ (`/tasks`) ก่อน route ที่มีพารามิเตอร์ (`/{docNo}`) และ `docNo` เป็น `YYYY/xxxxx` จึงต้อง `encodeURIComponent` ทุกครั้งที่ประกอบ URL
- เส้นที่ ไม่ใช่ของ SGI ห้ามใส่ prefix และห้ามแตะ — `GET /store/search` · `GET /store/all-regions` · `GET /common/common-code` · `GET /menus` · `GET /groups/current-user/permissions` · `POST /statement/upload-file-aws` · `GET /api/workflow/pending` เป็นของระบบ SBP เดิม
- BFF ส่งต่อทั้ง prefix (`/api/v1/sgi/*`) โดยไม่ตัดคำ · สิทธิ์เมนูผูกกับ URL ของ หน้าจอ (/sgi/<กลุ่ม>/...) ไม่ใช่ URL ของ API

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	ALL /api/v1/sgi/*; GET /api/v1/sgi/*?page=1&size=20; POST /api/v1/sgi/document/{docNo}/actions
Progress	Bootstrap env and API client; Login or restore session with refresh token; Load GET /auth/profile + GET /users/current + GET /menus + GET /groups/current-user/permissions (ทั้งหมดเป็นของระบบเดิมผ่าน BFF); Render routes/sidebar from menu contract
Output	ไม่มีตารางที่เอกสารนี้เขียนเอง — output คือ response ตาม envelope กลาง `{success, data}` และร่องรอยที่ตรวจย้อนได้ (log / sgi_consideration_logs / workflow_history ของ engine)

5.90 Integration Contracts Component Contract

ID	Component / Scope	Single responsibility	Definition of done
C01	Shared API client contract	สร้าง shared API client ตัวเดียวสำหรับ base URL, trace header, timeout และ response envelope	ทุก feature import client กลางและไม่มี axios/fetch instance แยก
C02	Auth/JWT consumption from platform reference	อ่าน access token จาก platform auth store, แนบ Bearer token และทำ refresh แบบ single-flight	401 พร้อมกัน refresh ครั้งเดียว, replay request เดิม และไม่สร้างหน้า Login ใหม่
C03	Error display and validation message mapping	แปลง HTTP/Axios failure เป็น ApiError พร้อม code, message, fieldErrors และ traceId โดยไม่แก้ข้อความจาก BE	validation banner/inline error แสดงข้อความและ traceId จาก response ได้ครบ
C04	Date/year/money/docNo formatting	ให้ formatter กลางสำหรับวันที่ (ค.ศ.), เดือน, เงิน, percent และ docNo โดยไม่เปลี่ยนค่าที่ส่ง API	payload และ UI ใช้ ค.ศ. เป็นค่าเริ่มต้น (buddhistEra=false); รูปแบบเงิน/docNo ตรงกันทุกหน้า
C05	Pagination, list empty/loading/error state	กำหนด PageResponse<T> และ state loading/empty/error/retry สำหรับ list ทุกชนิด	DataTable/Pager รักษา page/filter เดิมและไม่มี list shape เฉพาะหน้า
C06	Document action result enum and response consumption	กำหนด typed action request/response และ consume statusCode/nextSection ที่ BE คำนวณ	FE ส่งเฉพาะ result/comment และไม่มี client-side workflow routing
C07	RBAC/menu gating and editable section flags	สร้าง sidebar, route guard, visibleSections, editableSections และ actionOptions จาก platform/menu API	ไม่ hardcode RBAC role เป็นสิทธิ์เมนูหรือ section ที่แก้ไขได้

5.91 Integration Contracts API Adapter Map

Endpoint	Typed adapter purpose	Invoked by
ALL /api/v1/sgi/*	Error contract กลางสำหรับ FE ทุกหน้า	Attach token (ทุก API call)
GET /api/v1/sgi/*?page=1&size=20	List/pagination contract กลาง	Refresh token (401 non-auth endpoint)
POST /api/v1/sgi/document/{docNo}/actions	Document action contract ตัวอย่างเมื่อ currentSection=01 จึงเปลี่ยนไป 02; FE ห้ามส่งหรือคำนวณปลายทางเอง	Submit action (ปุ่มส่งดำเนินการ)

5.92 Integration Contracts Interaction State Machine

Action	Trigger	API / State transition	Expected visible result
Attach token	ทุก API call	shared/api/client.ts	Authorization header จาก auth store
Refresh token	401 non-auth endpoint	POST /api/v1/auth/refresh	single-flight แล้ว replay request เดิม

Action	Trigger	API / State transition	Expected visible result
Show API error	catch AxiosError	apiErrorMessage()	แสดงข้อความไทยจาก BE ตรง ๆ
Render list	GET list endpoint	PageResponse<T>	DataTable/Pager ใช้ shape เดียวกัน
Submit action	ปุ่มส่งดำเนินการ	POST /api/v1/sgi/document/{docNo}/actions	ส่ง `{result, comment}` และ consume `{statusCode,nextSection,message}`
Gate route/menu	login/bootstrap	GET /menus + GET /groups/current-user/permissions (ระบบเดิม ผ่าน BFF)	สร้าง sidebar และ route guard จาก menuCode

5.93 Integration Contracts Feature Failure Checks

Case	Feature-specific scenario	Expected evidence
FE-01	401 refresh single-flight	ไม่มี feature ใดสร้าง axios instance เอง
FE-02	403 route guard	ทุก API error แสดง message จาก BE โดยไม่ paraphrase
FE-03	error message passthrough	ทุก list endpoint ใช้ PageResponse shape เดียวกัน
FE-04	pagination pager mapping	วันที่ใน payload และหน้าจอเป็น ค.ศ. จาก formatter กลาง (แสดง พ.ศ. เฉพาะจุดที่เปิด flag)
FE-05	date BE display	Sidebar และ route access มาจาก GET /menus ของระบบเดิม ไม่ hardcode role
FE-06	action response invalidation	FE ไม่คำนวณ action routing เอง; ใช้ role profile และ actionOptions จาก API

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Attach token	ทุก API call	shared/api/client.ts	Authorization header จาก auth store
Refresh token	401 non-auth endpoint	POST /api/v1/auth/refresh	single-flight แล้ว replay request เดิม
Show API error	catch AxiosError	apiErrorMessage()	แสดงข้อความไทยจาก BE ตรง ๆ
Render list	GET list endpoint	PageResponse<T>	DataTable/Pager ใช้ shape เดียวกัน
Submit action	ปุ่มส่งดำเนินการ	POST /api/v1/sgi/document/{docNo}/actions	ส่ง `{result, comment}` และ consume `{statusCode,nextSection,message}`
Gate route/menu	login/bootstrap	GET /menus + GET /groups/current-user/permissions (ระบบเดิม ผ่าน BFF)	สร้าง sidebar และ route guard จาก menuCode

7. API Contract

ALL /api/v1/sgi/*

Error contract กลางสำหรับ FE ทุกหน้า

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	Endpoint has no JSON body/query object

Response


```
{
  "code": "VALIDATION",
  "message": "ข้อความภาษาไทยตรงตาม ข้อกำหนดทางธุรกิจ"
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
code	string	Yes	UTF-8; use value domain described by endpoint purpose
message	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/*?page=1&size;=20

List/pagination contract กลาง

Query Params


```
{
  "page": 1,
  "size": 20
}
```


Request Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	No	>= 1; default 1 · ● หน้าจอต้องส่งเลขหน้าที่เปลี่ยนจริง (ตารางตั้ง `lazy` + `totalRecords` + `onPage`) ห้ามขอทั้งชุดมาแบ่งหน้าเอง · ค่าค้นหรือตัวกรองเปลี่ยนเมื่อไร ต้องรีเซ็ตกลับหน้า 1 ไม่งั้นค้างอยู่หน้าที่ผลลัพธ์ใหม่ไม่มี
size	integer	No	1..100; default 20 · ● เพดาน 100 คือ `@Max(100)` ของ DTO ทั้ง BFF และ BE (BE ย้ำอีกชั้นด้วย `Math.min(size, 100)`) ส่งเกินได้ 400 แล้วตารางว่างทั้งหน้า — ตัวเลือกในช่อง Show ทุกค่าต้องไม่เกินค่านี้

Response


```
{
  "page": 1,
  "size": 20,
  "total": 0,
  "items": []
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	Yes	>= 1; default 1 · ● หน้าจอต้องส่งเลขหน้าที่เปลี่ยนจริง (ตารางตั้ง `lazy` + `totalRecords` + `onPage`) ห้ามขอทั้งชุดมาแบ่งหน้าเอง · คำค้นหรือตัวกรองเปลี่ยนเมื่อไร ต้องรีเซ็ตกลับหน้า 1 ไม่งั้นค้างอยู่หน้าที่ผลลัพธ์ใหม่ไม่มี
size	integer	Yes	1..100; default 20 · ● เพดาน 100 คือ `@Max(100)` ของ DTO ทั้ง BFF และ BE (BE ย้ำอีกชั้นด้วย `Math.min(size, 100)`) ส่งเกินได้ 400 แล้วตารางว่างทั้งหน้า — ตัวเลือกในช่อง Show ทุกค่าต้องไม่เกินค่านี้
total	integer	Yes	UTF-8; use value domain described by endpoint purpose
items	array<object>	Yes	JSON array; element type shown in Type column

POST /api/v1/sgi/document/{docNo}/actions

Document action contract ตัวอย่างเมื่อ currentSection=01 จึงเปลี่ยนไป 02; FE ห้ามส่งหรือคำนวณปลายทางเอง

Request

```
{
  "result": "เห็นควรชดเชย",
  "comment": "เห็นควรชดเชยตามหลักเกณฑ์"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
result	string	Yes	ช่อง ผลการพิจารณา (มีเฉพาะหน้า *ที่เกี่ยวข้อง*) · ● ตัวเลือกบนหน้าจอเหลือ 2 ค่า — `APPROVE` (ประกันรายได้) / `REJECT` (ไม่ประกันรายได้) (มติผู้ใช้ 2026-09-23 · กติกาเดียวกับตัวกรองของหน้ารายงาน อ่านจากไฟล์กลาง `constants/sgi/considerationResult`) · ▲ โดเมนของ API ยังรับครบ `APPROVE` / `REJECT` / `CANCELLED` / `PENDING` / `NONE` ห้ามลด — `CANCELLED`/`PENDING` ยังเกิดจริงในข้อมูลและถูกเก็บที่ `sgi_consideration_logs.result_category` การจำกัดนี้อยู่ที่ตัวเลือกบนหน้าจอเท่านั้น · ดูจากผลพิจารณา ล่าสุด ของเอกสารใน `sgi_consideration_logs` ไม่ได้อยู่ที่หัวเอกสาร
comment	string	No	trimmed UTF-8 Thai text; required by operation/business rule

Response

```
{
  "statusCode": "02",
  "nextSection": "02",
  "message": "submitted"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
statusCode	string	Yes	canonical code; do not replace with display label
nextSection	string	Yes	canonical code; do not replace with display label
message	string	Yes	UTF-8; use value domain described by endpoint purpose

8. Skeleton Code (โครงโค้ดตั้งต้นของหน้าจอนี้)

โค้ดชุดนี้อิง convention ของ portal เดิม `srm-sps-spsap-web-frontend` (build target `sbpm`): Next.js App Router + `'use client'`, PrimeReact ที่ห่อไว้แล้วใน `@/components/Form` และ `@/components/Table`, react-hook-form + yup, Zustand `permissionStore`, axios instance กลาง `@/lib/apiClient` และ react-query 5 — โปรเจกต์ไม่มี chart library จึงไม่มีโค้ดกราฟในเอกสารนี้ คัดลอกไปตั้งต้นได้ทันที แล้วเติมจุดที่กำกับ `TODO:`

8.1 ผังไฟล์ที่ต้องสร้าง

โครงไฟล์อิง portal เดิม (`srm-sps-spsap-web-frontend`, target `sbpm`) — โมดูล SGI อยู่ใต้ `src/app/(main)/sgi/*` และ import ผ่าน alias `@/*` ทุกจุด

Path ไฟล์	หน้าที่
src/types/sgi/common.ts	types — ApiResponse / PageResponse / ApiError กลางของโมดูล
src/lib/sgi/apiError.ts	helper — แปลง AxiosError เป็นข้อความไทยจาก BE (ไม่ paraphrase)
src/utils/sgi/format.ts	helper — formatMonthThai / formatAmount / docNo (ค.ศ. ทั้งหมด · ไม่แปลง พ.ศ.)
src/services/sgi/integration.service.ts	service — เรียก BFF ผ่าน apiClient (POST)
src/hooks/sgi/integration.query.ts	hook — query key factory + useQuery/useMutation + invalidate
src/types/sgi/integration.ts	types — request/response ตาม API contract ของเอกสารนี้

8.2 types/helper กลาง (envelope, error message, formatter)

```
// src/types/sgi/common.ts — สัญญากลางที่ทุกหน้าในโมดูล SGI ใช้ร่วมกัน
// envelope ต้องตรงกับ store-backend: { success, data } / { success:false, data:null, error:{code,message} }

export interface ApiError {
  code: string; // เช่น VALIDATION, ACTION_RESULT_REQUIRED
  message: string; // ข้อความไทย verbatim จาก BE — ห้าม paraphrase ฝั่ง FE
}

export interface ApiResponse<T> {
  success: boolean;
  data: T;
  error?: ApiError | null;
}

export interface PageResponse<T> {
  page: number; // >= 1
  size: number; // <= 100
  total: number;
  items: T[];
}

/** payload ของทุก workflow action — FE ส่งได้แค่ 2 field นี้ ห้ามส่ง nextSection เอง */
```

```
export interface DocumentActionRequest {
  result: string; // ต้องเป็นค่าจาก actionOptions ที่ API ส่งมาเท่านั้น
  comment: string;
}

export interface ActionResponse {
  statusCode: string;
  nextSection: string | null;
  message: string;
}

// ---------------------------------------------------------------------------
// src/lib/sgi/apiError.ts
// ---------------------------------------------------------------------------
import { AxiosError } from 'axios';

export function apiErrorMessage(error: unknown): string {
  const message = (error as AxiosError<{ error?: ApiError }>)?.response?.data?.error?.message;
  if (message) return message; // ใช้ข้อความจาก BE ตรง ๆ
  return 'ไม่สามารถเชื่อมต่อระบบได้ กรุณาลองใหม่อีกครั้ง'; // fallback เฉพาะ network/no-response
}

// ---------------------------------------------------------------------------
// src/utils/sgi/format.ts — formatter กลางจุดเดียว · ค.ศ. ทั้ง payload และ display (มติ 2026-08-06)
// ---------------------------------------------------------------------------
export const formatMonthThai = (isoMonth: string): string => {
  const [year, month] = isoMonth.split('-');
  return `${month}/${Number(year) + 543}`;
};

export const formatAmount = (value: number): string =>
  value.toLocaleString('th-TH', { minimumFractionDigits: 2, maximumFractionDigits: 2 });

// TODO: ยืนยันรูปแบบวันที่/เดือนกับ ข้อกำหนดทางธุรกิจ ก่อนใช้จริง (บางหน้าจอแสดง ค.ศ. ตามระบบ SBP เดิม)
```

8.3 service — `src/services/sgi/integration.service.ts`

```
// src/services/sgi/integration.service.ts
// apiClient = axios instance กลาง (baseURL = bffUrl ซึ่งรวม /api/v1 แล้ว, withCredentials, refresh-token interceptor, global loading)
// ห้ามสร้าง axios instance ใหม่ และห้าม set Authorization header เอง — session อยู่ใน httpOnly cookie ของ BFF

import apiClient from '@/lib/apiClient';
import type { ApiResponse } from '@/types/sgi/common';
import type * as T from '@/types/sgi/integration';

/** POST /api/v1/sgi/document/{docNo}/actions — Document action contract ตัวอย่างเมื่อ currentSection=01 จึงเปลี่ยนไป 02; FE
ห้ามส่งหรือคำนวณปลายทางเอง */
export async function createSgiDocumentActions(docNo: string, body: T.CreateSgiDocumentActionsRequest):
Promise<T.CreateSgiDocumentActionsResponse> {
  const { data } = await
apiClient.post<ApiResponse<T.CreateSgiDocumentActionsResponse>>(`/sgi/document/${encodeURIComponent(docNo)}/actions`, body);
  return data.data;
}

// ── envelope: อ่าน `data.data` ชั้นเดียว (ยืนยันจากโค้ดจริงของ BFF 2026-09-04) ──
// BFF มี ResponseInterceptor ระดับ global (src/common/interceptors/response.interceptor.ts)
// ที่หุ้อผลลัพธ์ของ controller เป็น { success, data, requestId } ให้เสมอ
// ▲ ถ้า client service ของ BFF คืน `response.data` ดิบ (= envelope ของ store-backend)
// interceptor จะเห็นคีย์ success แล้วห่อซ้ำ → FE ได้ { success, data: { data: <payload> } }
// สัญญาที่ตกลง: **BFF ต้อง unwrap ของ store-backend ก่อน 1 ชั้น** (คืน response.data.data)
// FE จึงอ่าน data.data ชั้นเดียวตามโค้ดด้านบน · requestId ใช้อ้างอิงตอนแจ้งปัญหา
```

8.4 types — `src/types/sgi/integration.ts`

```
// src/types/sgi/integration.ts — ตรงกับตาราง API ในเอกสารนี้
// วันที่/เดือนเป็น ค.ศ. ทั้ง payload (ISO) และ display — ไม่แปลงเป็น พ.ศ. (มติ 2026-08-06)

/** POST /api/v1/sgi/document/{docNo}/actions — request */
export interface CreateSgiDocumentActionsRequest {
  result: string;
  comment: string;
}

/** POST /api/v1/sgi/document/{docNo}/actions — response */
export interface CreateSgiDocumentActionsResponse {
  statusCode: string;
  nextSection: string;
  message: string;
}

// TODO: ใส่ nullable / required ให้ตรงกับ contract ฉบับล่าสุดของ BE
```

8.5 react-query keys + hooks — `src/hooks/sgi/integration.query.ts`

```
// src/hooks/sgi/integration.query.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';
import * as api from '@/services/sgi/integration.service';
import type * as T from '@/types/sgi/integration';

export const integrationKeys = {
  all: ['sgi', 'integration'] as const,
};

export function useCreateSgiDocumentActionsMutation(docNo: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (body: T.CreateSgiDocumentActionsRequest) => api.createSgiDocumentActions(docNo, body),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: integrationKeys.all }); // reload list/detail/timeline
    },
    // TODO: onError -> แสดง apiErrorMessage(error) ผ่าน Toast กลาง
  });
}
```

- ทุกหน้าเช็คสิทธิ์ด้วย `permissionStore.hasPermission(url, 'canView'|'canManage'|'canExport'|'canOther')` แล้ว render `<AccessDenied />` เมื่อไม่มีสิทธิ์
- เมนู/สิทธิ์มาจาก `GET /menus` และ `GET /groups/current-user/permissions` — ห้าม hardcode role หรือรายการเมนูใน FE
- session อยู่ใน httpOnly cookie ของ BFF (`withCredentials: true`) — FE ไม่เก็บและไม่แนบ token เอง
- payload และการแสดงผลใช้วันที่ ค.ศ. เสมอ ผ่าน formatter กลางจุดเดียว — ไม่แปลงเป็น พ.ศ. (มติ 2026-08-06)
- ข้อความ error แสดงจาก `error.message` ของ BE ตรง ๆ (ห้าม paraphrase) — fallback ใช้เฉพาะกรณี network error

9. Processing Flow

Step	Description
1	Bootstrap env and API client
2	Login or restore session with refresh token
3	Load GET /auth/profile + GET /users/current + GET /menus + GET /groups/current-user/permissions (ทั้งหมดเป็นของระบบเดิมผ่าน BFF)
4	Render routes/sidebar from menu contract
5	All feature hooks use shared API client and PageResponse/Error types
6	Document action sends `{result, comment}` only and consumes `{statusCode,nextSection,message}`
7	All display formatting goes through shared/lib/format.ts

10. Acceptance Criteria

- ไม่มี feature ใดสร้าง axios instance เอง
- ทุก API error แสดง message จาก BE โดยไม่ paraphrase
- ทุก list endpoint ใช้ PageResponse shape เดียวกัน
- วันที่ใน payload และหน้าจอเป็น ค.ศ. จาก formatter กลาง (แสดง พ.ศ. เฉพาะจุดที่เปิด flag)
- Sidebar และ route access มาจาก GET /menus ของระบบเดิม ไม่ hardcode role
- FE ไม่คำนวณ action routing เอง; ใช้ role profile และ actionOptions จาก API

11. Developer Test Checklist

No	Test
1	401 refresh single-flight
2	403 route guard
3	error message passthrough
4	pagination pager mapping
5	date BE display
6	action response invalidation
7	menu filtering by API